Bayesian-Optimized Variational Quantum Circuits

Teaching a 2-qubit circuit to classify data, without gradients

Jialun Li Juan Vera

PHYS 126

October 14, 2026

The question

Can a **tiny quantum circuit** learn to classify data as well as a **small neural network**...

...if we train it without gradients?

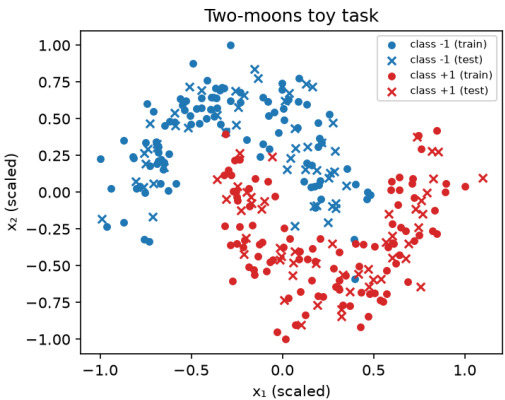
Why "without gradients"?

- Neural nets are trained by following the slope of the error.
- On a real quantum computer, that slope is *expensive and noisy* to measure.
- So people want optimizers that only need the score, not the slope.

What we did

- Built a 2-qubit circuit in Qiskit.
- Trained it with **Bayesian optimization**.
- Raced it against classical neural networks on the same task.

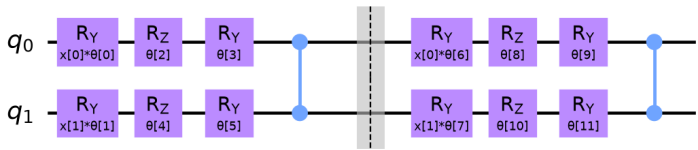
The task: two moons



- 300 points in two interleaved crescents, red and blue.
- Goal: given a new point, say which colour it is.
- 200 points to learn from, 100 held back to test on.

No straight line separates them. A straight-line classifier scores 84 %. Everything above that needs a *curved* boundary.

What a quantum classifier is



two of the four layers shown

1. Rotate each qubit by an angle that depends on the data point (x_1, x_2).
2. Rotate again by **trainable** angles θ (the "knobs").
3. Entangle the two qubits. Repeat four times.
4. Measure qubit 0.

Output: a number $f \in [-1, 1]$.
$f > 0 \Rightarrow$ red, $f < 0 \Rightarrow$ blue.

24 knobs in total. **Training** = turning the knobs until the sign is right for as many points as possible.

Why we can't just use gradients

Neural network

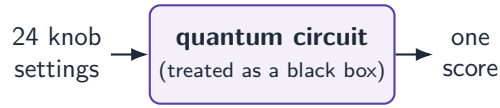
- Backpropagation: one backward pass gives the slope for *every* weight.
- Cost $\approx$ one forward pass.

Quantum circuit

- You can't look inside the state without destroying it, so there is no backward pass.
- The slope for one knob costs two extra full runs:

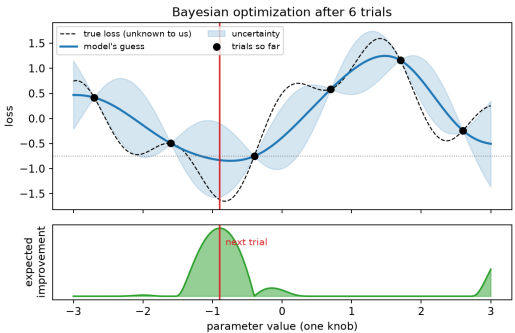
$$\frac{\partial f}{\partial \theta} = \tfrac{1}{2}\left[f\left(\theta + \tfrac{\pi}{2}\right) - f\left(\theta - \tfrac{\pi}{2}\right)\right]$$

- 24 knobs $\Rightarrow$ **48 extra runs** per data point



So we treat it as a black box: knobs in, score out. We need an optimizer that works from scores alone.

Bayesian optimization in plain words



You can only afford a few hundred trials. After each one:

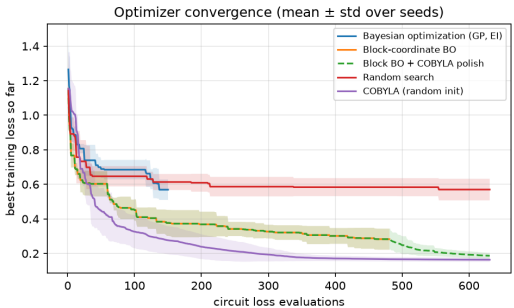
1. Fit a smooth guess through every result so far, *with error bars*.
2. Try next wherever the guess is low **or** the error bars are wide.

That trade-off has a name, *Expected Improvement*:

$$\mathrm{EI} = \underbrace{(\mathcal{L}^* - \mu)\,\Phi(z)}_{\text{probably good}} + \underbrace{\sigma\,\varphi(z)}_{\text{haven't looked}}$$

Every trial is chosen by the model, not by a fixed rule.

First attempt: Bayesian optimization on all 24 knobs



- Blue: BO over all 24 knobs at once, 150 trials. Red: 24 knob settings picked **completely at random**.

They overlap. Bayesian optimization was no better than guessing. Test accuracy 79 % for both.

Why? With 24 knobs the space is enormous. 150 trials is far too few for the model to learn its shape, so its guess stays flat and "try where the error bars are wide" means "try anywhere". More trials and other selection rules did not help.

Fix 1: tune six knobs at a time

The idea

- The circuit has four layers of six knobs each.
- Freeze three layers. Run Bayesian optimization on the **six knobs** of the remaining one, 40 trials.
- Move to the next layer. Go round all four layers three times.

The analogy

Tuning a guitar one string at a time, then going round again, instead of turning all six pegs at once.

Why it works

- In six dimensions, 40 trials is enough for the model to actually learn the shape of the landscape.
- We only ever accept a change that improves the score, so it can never get worse.

Same optimizer, same kind of budget: accuracy jumps from 79 % to 93 %.

Still a bit unlucky on some seeds (88 to 97 %): it can get stuck where no *single* layer can improve things.

Fix 2: finish with a local “polish” (COBYLA)

What COBYLA is

- A classic optimizer from 1994. **Not** Bayesian: no model, no error bars.
- Looks at the scores of a few nearby knob settings, fits a *flat plane* through them, and steps toward the low side.
- If the step helped, take another. If it didn’t, take smaller steps.

The analogy

Walking downhill in fog: feel the slope under your feet, step that way, shorten your stride when you start going up.

How we used it

- Start from the best setting Bayesian optimization found.
- 150 small downhill steps.

Bayesian optimization finds the right valley. COBYLA walks to the bottom of it. Result: 96 % on every seed.

Global search first, local search second. This is a standard pairing.

Two sanity checks (the “controls”)

A result only means something if you know what the dumb alternatives would have scored. Same budget of circuit runs for all.

Control 1: random guessing

- Pick 24 random knob settings, score them, keep the best. Repeat 631 times.
- Scored **79 %**.
- Tells us whether an optimizer is doing anything at all.

Control 2: COBYLA on its own

- The same downhill walker, but starting from a *random* point, with no Bayesian optimization first.
- Scored **96 %**, just like our full pipeline.
- Tells us whether the Bayesian part was actually needed.

Control 2 is the uncomfortable one: on our noise-free simulator, the landscape was smooth enough that simply walking downhill from anywhere worked.

The classical competitors

Three models, from simplest to biggest:

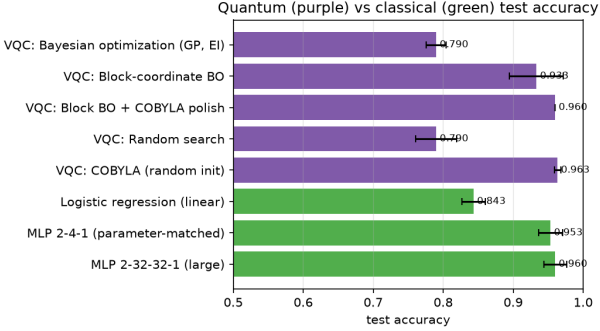
- **A straight line** (3 weights). The baseline that *can't* curve.
- **A tiny neural network** (17 weights). Chosen to match our circuit's 24 knobs as closely as possible. This is the fair fight.
- **A big neural network** (1185 weights). Shows the ceiling when size is no object.

How they were trained

- With gradients, the way neural nets normally are: backpropagation computes the slope of the error for every weight at once.
- An optimizer called L-BFGS then follows that slope, using the curvature it picks up along the way to take smart, long steps.
- Converges in under a second.

The classical models get every advantage. Can the gradient-free circuit keep up?

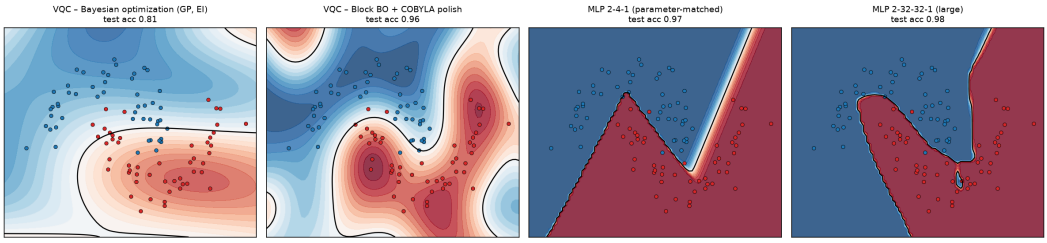
Results



1. **The quantum circuit reaches 96 %**, the same as the neural networks.
2. BO over all 24 knobs at once was **no better than random guessing** (79 %).
3. BO on 6 knobs at a time fixed it (93 %); the polish made it reliable (96 %).
4. COBYLA alone also reached 96 %.

3 random seeds, 100 test points: a one-point gap is noise. Still 95 % when read out with only 256 measurement shots per point, as on real hardware.

What the models learned



Left to right: the boundary goes from a blob (BO on all 24 knobs) to a curve that hugs the moons (BO on 6 at a time + polish).

The quantum boundary is **smooth and wavy**; the neural net's is made of **straight segments**. Same problem, different shapes.

Conclusions

1. A 2-qubit circuit with 24 knobs, trained with **no gradients at all**, matched classical neural networks with 17 and 1185 weights: 96 % on every seed.
2. Bayesian optimization **cannot handle all 24 knobs at once**. It only works when restricted to a handful of knobs at a time, and a short local polish makes it reliable.
3. On a noise-free simulator, the simpler downhill walker COBYLA did just as well on its own. Bayesian optimization's real advantages are in noisy, expensive-to-evaluate settings, which our simulator does not create.
4. The trained circuit survives realistic measurement: 95 % with 256 shots per point.

Not a quantum advantage, and none expected at 2 qubits. The point was showing which training methods work when gradients are off the table.

Code, results and a live terminal demo: `bo-vqc` repository.